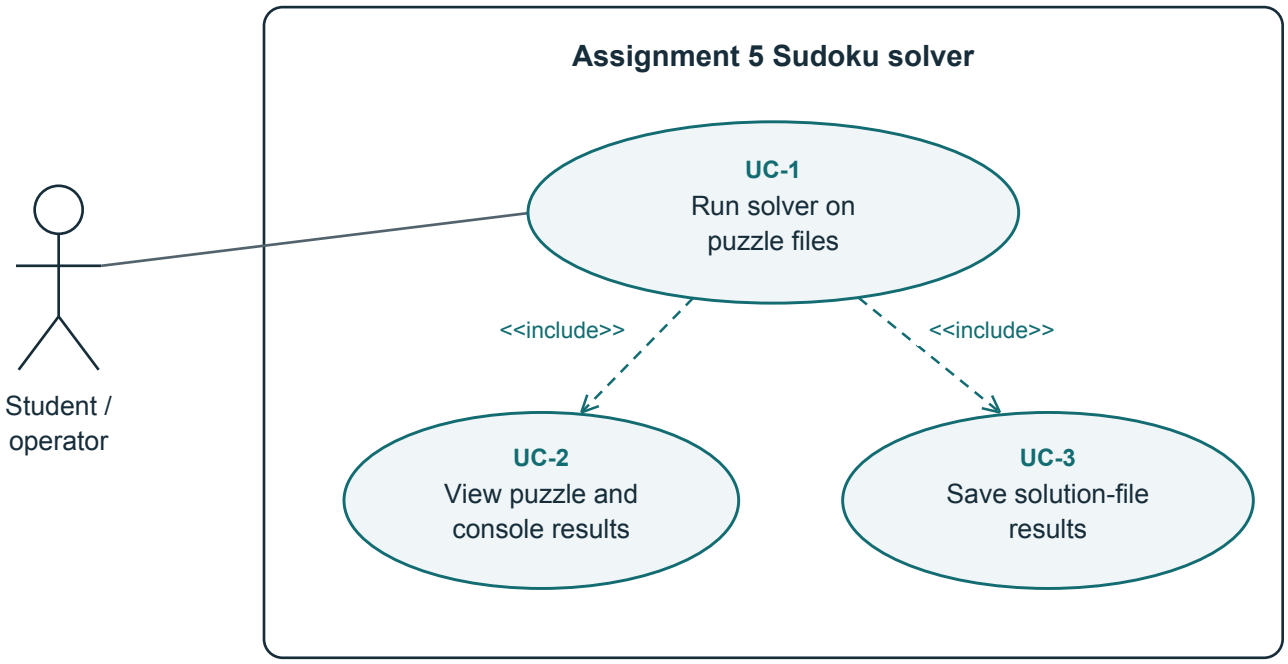
01 / USE-CASE MODEL

Run, review, and save

The student runs the solver on a set of puzzle files. For each readable puzzle, the program shows the puzzle and its results and writes the results to a solution file.



ID	Identified use case	Brief description
UC-1	Run solver on puzzle files	Choose the default batch or command-line filenames and process each file in order.
UC-2	View puzzle and console results	See the filename, original grid, every solution and the total, or the relevant no-solution/error message.
UC-3	Save solution-file results	Write every solution and its count, or the required no-solution line, to the matching output file.

How to read the diagram

The solid line connects the actor to the entry goal. Dashed <<include>> arrows point from UC-1 to its included behavior, UC-2 and UC-3. These are required parts of normal processing, not optional menu choices or an order-of-calls diagram. An error can end the current file before later steps are reached.

02 / REVISION OF WESTON'S DIAGRAM

What stays, what changes

This revision of Weston G. Uptmor's Assignment 4 diagram keeps the original run-and-display goals and adds the required saved results.

Original diagram	Assignment 5 revision
Student; Run solver on puzzle files	Keep the external student/operator and the same entry goal as UC-1.
Load puzzle file	Keep loading and its failure path in UC-1's flow. The puzzle file is input data, not an independent actor.
Print original puzzle; Print solution(s); Report no solution	Preserve all three console outcomes in UC-2. Add the filename and final count explicitly.
Search for solutions; Record solution	Retain exhaustive search in UC-1's flow. Each solution is shown in UC-2 and written to a file through the new UC-3.
Find next empty cell; Try digits 1 to 9; Validate placement; Check row/column/3x3 block; Backtrack	Keep these as internal search steps, not separate actor goals. The use-case diagram is not a method-call graph.
Puzzle files and Console shown as actors	Describe them as inputs and output channels. Only the student/operator is modeled as an external actor.

Scope and actual program behavior

- With no arguments, main processes puzzle1.txt through puzzle5.txt. With arguments, it uses only those filenames, in the given order. There is no interactive prompt or command-line solution-limit option.
- The normal run has no solution-count cap. It uses recursive backtracking to find all solutions; a puzzle with many solutions can take a long time and produce large output.
- Output files are created in the process's working directory, not beside the input file. The first digit group in the input's basename supplies N in solutionN.txt; a name without digits uses solution.txt.
- Opening an output file replaces its previous contents. Inputs that map to the same output name can overwrite each other's results. The program does not create unique backup names.

Authorship and sources

Original diagram: Weston G. Uptmor, Assignment4UseCases-1.pdf. Revised by Francis Tryban with OpenAI GPT-6.1-Sol in Zed. Creation date: October 29, 2026. Sources: Assignment 5, instruction 7(a–d), and the current Assignment5.cpp.

03 / UC-1 · DETAILED DESCRIPTION

Run solver on puzzle files

Goal: process a batch of Sudoku puzzles and provide console and file results for each puzzle that can be processed.

Primary actor	Student/operator.
Trigger	The student launches Assignment5.exe, with or without puzzle filenames.
Preconditions	The executable is available and the process can start. Readable input and writable output are needed for success, but are not assumed for every file.
Success result	Every selected valid puzzle is fully searched, its results are displayed, and its output file is completed. With no solutions, both channels report that outcome.

Main flow · step by step

01 The student starts the program. With no filename arguments, the program selects puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt and puzzle5.txt. Otherwise it selects each argument as an input filename.

02 For the next selected filename, the program starts the console report with a separator and the filename (UC-2).

03 The program opens and loads the puzzle. A successful load supplies 81 cells; digits 1–9 are givens, and _, . or 0 can mark blanks. It displays the original puzzle on the console.

04 The program derives the output name and opens the result file (UC-3). It does not start solving if that open fails.

05 The program checks whether the givens conflict. If they are consistent, it searches the empty cells recursively. It tries digits 1–9, checks the row, column and 3×3 block, then undoes a choice when returning to try another path.

06 Each complete solution is numbered and printed on the console (UC-2), and its spaced grid is written immediately to the output file (UC-3). The search continues until all possible solutions have been explored.

07 The program prints the total or no-solution message, writes the corresponding file summary, closes the output file and checks for a write failure.

08 The program repeats steps 2–7 for each remaining filename. After the list is exhausted, main returns 0.

Alternatives and errors

- At step 3, a failed open or rejected puzzle load produces the console read-error message. No output file is opened or newly created; an existing output file is left untouched. Continue with the next input.
- At step 4, a failed output open produces the create-error message and skips solving that puzzle. At step 7, a late write/close failure produces the finish-writing error after the normal console result. Both paths continue with the next input.
- Conflicting givens or a search with no completions yield zero solutions, not a read error. UC-2 and UC-3 report no solution. Errors do not cause main to return a nonzero status.

04 / UC-2 · DETAILED DESCRIPTION

View puzzle and console results

Goal: show the filename, starting grid, all solutions and any error. In the messages below, <output-name> means the actual derived filename, without angle brackets.

Primary actor	Student/operator, reading standard output.
Trigger	UC-1 begins processing a selected input file.
Preconditions	The program is running. Showing the original grid requires a successful load; showing solutions also requires a successful output-file open.
Success result	The console shows the filename, original puzzle, every solution and a count, or a no-solution result. File-operation errors are also reported on the console.

Main flow · step by step

01 The program prints a separator and Puzzle file: followed by the input filename.

02 After a successful load, it prints Puzzle: and the original 9×9 grid. Blank cells appear as underscores. The console grid has spaces and 3×3 box dividers.

03 For each solution found, the program prints Solution 1:, Solution 2:, and so on, followed by that completed grid in the same console style.

04 When the exhaustive search finishes with n > 0, the program prints n solution found. for one solution or n solutions found. for more than one.

05 The student can read the report. The program moves to the next input without waiting for confirmation.

Alternatives and errors · exact console text

Condition	Visible result and continuation
Input cannot be loaded	Error: could not read a valid 9x9 puzzle from this file. No original grid or solutions are printed for that file.
Output cannot be opened	Error: could not create <output-name>. The filename and original grid have already been shown. No search or solution count follows for that file.
No solutions	No solution found This console line has no final period. No Solution k: grids appear.
Late write/close failure	Error: could not finish writing <output-name>. The normal console solutions/count or no-solution message have already been printed.

05 / UC-3 · DETAILED DESCRIPTION

Save solution-file results

Goal: leave a plain-text results file for each successfully processed puzzle, containing every solution or the required no-solution message.

Primary actor	Student/operator, who receives the saved results through UC-1. Saving is automatic, not a separate command.
Trigger	UC-1 has loaded and displayed the current puzzle and attempts to prepare its output file.
Preconditions	The input was loaded. Successful completion also requires that the output can be opened and all writes and the close succeed.
Success result	The derived output file contains all completed grids followed by Solutions found: n, or exactly the single line No solution found. when n is zero.

Main flow · step by step

01 The program takes the first consecutive group of digits in the input basename. puzzle3.txt becomes solution3.txt; folder7/puzzle12.txt becomes solution12.txt. Without digits, the name is solution.txt.

02 It opens that output name in the working directory, replacing any existing contents. The solver starts only if this open succeeds.

03 As each solution is found, it writes nine rows of nine digits, separated by one space. Each row ends with a newline; a blank line follows each completed grid. There are no box dividers or Solution k: labels in the file.

04 After all solutions are found, it writes Solutions found: n and a newline, where n is the total number of grids written.

05 It closes the stream and checks its state. If the writes and close succeeded, the file is complete. UC-1 proceeds to the next input.

Alternatives and errors

- Zero solutions: write only No solution found. followed by a newline. Do not write a grid or Solutions found: 0. This also applies when the givens conflict.
- Failed output open: report Error: could not create <output-name>. on the console and skip solving. No completed result file is promised.
- Late write/close failure: the search and normal console reporting still finish. Then report Error: could not finish writing <output-name>. The file may be incomplete; the program does not retry, restore its old contents or remove a partial file. Continue with the next input.

Exact file endings

With solutions: all spaced 9×9 grids, each followed by a blank line, then Solutions found: n. With none: the single line No solution found. The period belongs only to the no-solution sentence. Both forms end with a newline. A failed input load never reaches this use case and does not alter an existing result file.